

MANUTENIBILIDADE

Como projetar software que continua fácil de mudar

Manutenabilidade • SOLID • Design Patterns • Métricas

“Quanto custa fazer uma mudança pequena em um sistema grande?”

A manutenibilidade aparece no esforço necessário para entender, modificar, testar e corrigir software.

O que é manutenibilidade?

Entender

Encontrar rapidamente onde uma regra ou comportamento está implementado.

Modificar

Fazer a mudança sem espalhar alterações pelo sistema.

Testar

Conseguir validar a mudança com baixo esforço.

Corrigir

Diagnosticar e resolver defeitos sem efeitos cascata.

Objetivo: tornar mudanças futuras previsíveis, seguras e baratas.

Por que sistemas ficam difíceis de manter?

Acoplamento

Mudanças reverberam em muitas partes

Complexidade

Há caminhos demais para entender e testar

Duplicação

A mesma regra existe em vários lugares

Dependências frágeis

Alterações exigem conhecer detalhes internos

As consequências da baixa manutenibilidade



Efeito econômico: o custo de cada mudança aumenta.

O que queremos no design?

Baixo acoplamento

Poucas dependências entre módulos.

Alta coesão

Responsabilidades relacionadas permanecem juntas.

Responsabilidades claras

Cada parte tem um propósito compreensível.

Testabilidade

Comportamentos podem ser verificados isoladamente.

Mudanças localizadas

Alterações afetam poucas partes do sistema.

Dependências explícitas

É fácil enxergar de onde vem cada colaboração.

Como medir manutenabilidade?

Complexidade

Cyclomatic Complexity
Cognitive Complexity

Estrutura

Coupling
Cohesion
Dependency Cycles

Código

Duplicação
Code Smells
Tamanho

Mudança

Lead Time
Change Failure Rate
Defeitos

Nenhuma métrica isolada responde: “este sistema é manutenível?”

Ferramentas de Manutenibilidade

Código bem feito

Clean Code
SOLID
Padrões de Projeto

Testes

Testes de Unidade
Testes de Integração
Testes End-to-End

Análise de Código

Bad Smells
Falhas de Segurança

Observabilidade

Métricas de Engenharia de Software

Nenhuma métrica isolada responde: “este sistema é manutenível?”

PARTE 2

SOLID

Cinco princípios para pensar o design com foco em mudança, responsabilidades e dependências.

SOLID: não é o objetivo, é uma ferramenta

S

Single Responsibility

Mudanças relacionadas ficam localizadas.

O

Open/Closed

Adicionar comportamento exige menos alteração do existente.

L

Liskov Substitution

Implementações substitutas preservam expectativas.

I

Interface Segregation

Dependências não precisam carregar métodos irrelevantes.

D

Dependency Inversion

Código depende de abstrações, não de detalhes.

S — Single Responsibility Principle

Uma classe/módulo deveria ter uma responsabilidade coesa — e, portanto, um conjunto coerente de motivos para mudar.

ANTES

OrderService

- calcula preço
- salva no banco
- envia e-mail
- gera PDF

DEPOIS

Order
PricingService
OrderRepository
NotificationService

Pergunta: “quantas razões diferentes podem fazer esta classe mudar?”

O — Open/Closed Principle

O sistema deve permitir extensão sem transformar cada nova regra em uma alteração espalhada.

PaymentService → Strategy → Pix / Card / Boletto

NOVO COMPORTAMENTO

+ ApplePayStrategy

O ponto não é “nunca alterar código”. É reduzir o raio de impacto de uma nova regra.

L, I e D: substituição, interfaces e dependências

L — Liskov

Substituições precisam respeitar o contrato esperado.

Sinal de alerta: subclasses que exigem “tratamento especial”.

I — Interface Segregation

Evite interfaces “Deus”.

Quem depende de uma interface deve depender apenas do que usa.

D — Dependency Inversion

Detalhes externos devem depender de abstrações estáveis.

Ajuda a isolar banco, API, fila e frameworks.

PARTE 3

DESIGN PATTERNS

Soluções recorrentes para problemas recorrentes de design.

Pattern 1 — Strategy

Problema: lógica condicional cresce cada vez que surge um novo comportamento.

```
1 if (method == PIX)
    System.out.println("Faz o pix");
2 if (method == CARD)
    System.out.println("Faz o pagamento
com cartao");
3 if (method == BOLETO)
    System.out.println("Manda o boleto");
```

PaymentStrategy

Pix

Card

Boleto

Conexão: Strategy + Open/Closed + testabilidade

Pattern 2 — Factory

Problema: criação de objetos pode ficar distribuído em várias partes do código.

```
1 if (method == PIX)
    new Pix();
2 if (method == CARD)
    new Card();
3 if (method == BOLETO)
    new Boleto();
```

PaymentFactory

Pix

Card

Boleto

Outros Padrões

- Padrões que já usamos:
 - Repository
 - Injeção de Dependência
 - Decorator
- Padrões que recomendo estudar:
 - Builder
 - Observable
 - Listener
 - Composite
 - Adapter

Conclusão

- 1 Manutenibilidade é sobre custo e risco de mudança.
- 2 Acoplamento, coesão, complexidade e testabilidade são propriedades-chave.
- 3 SOLID ajuda a raciocinar sobre responsabilidades e dependências.
- 4 Patterns são soluções recorrentes — não regras universais.
- 5 Métricas são essenciais, senão é impossível saber se a manutenibilidade é boa ou não.

Próximo passo da aula: refatorar um exemplo real e medir o antes/depois.